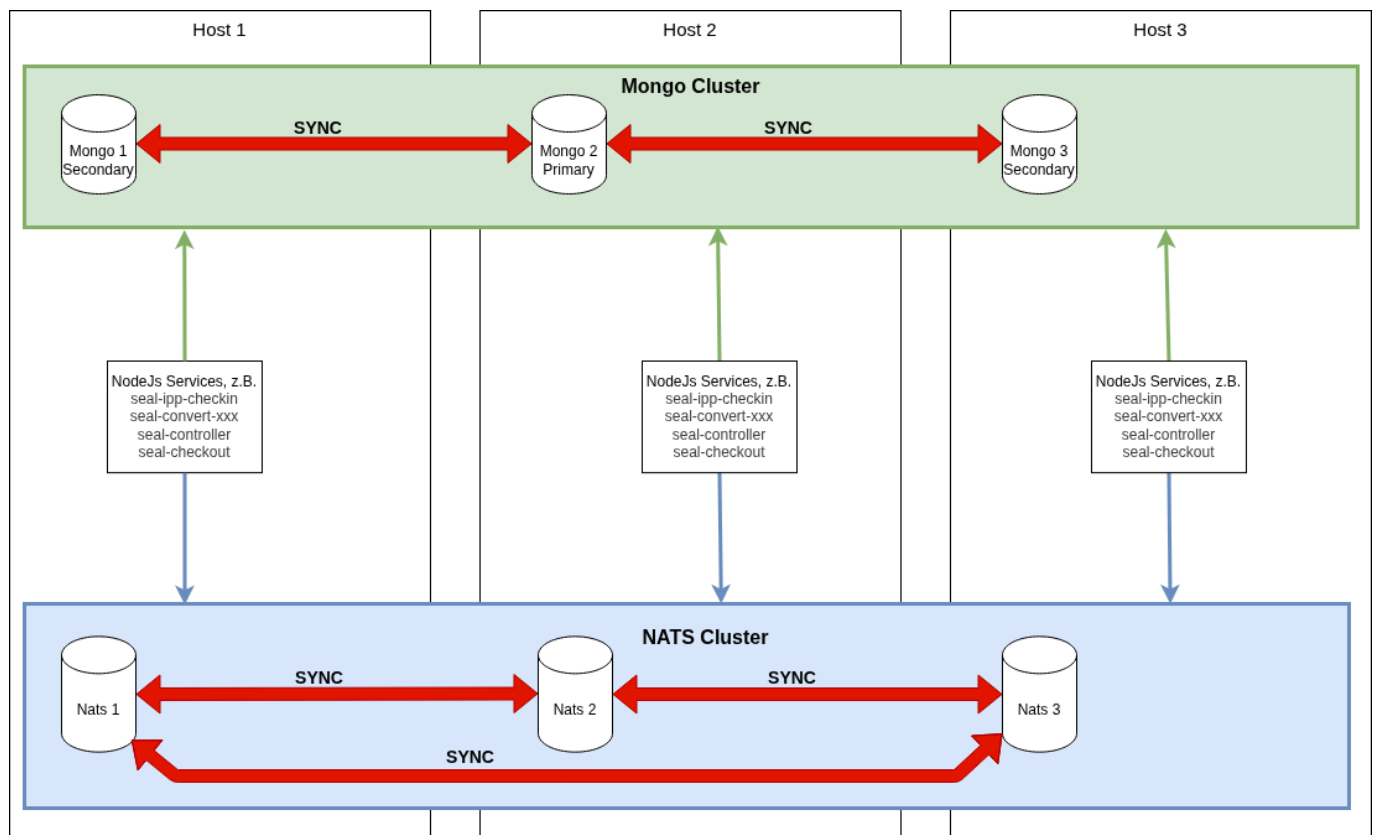


MongoDB & NATS

PLOSSYS Output Engine Architektur



MongoDB und NATS bilden die Infrastruktur unserer Software. Jeglicher Datenaustausch zwischen einzelnen NodeJs Services findet über die Infrastruktur-Dienste statt, unabhängig davon, ob ein Cluster oder ein einzelner Server zum Einsatz kommt.

MongoDB

MongoDB Config

Cache einschränken

Ein MongoDB zieht im Default "(Hauptspeichergröße / 2) - 1" GB RAM. Auf Rechnern mit wenig RAM kann der MongoDB Cache eingeschränkt werden. Erfahrungsgemäß sollten es aber **mindestens** 2GB sein.

```
storage:
  wiredTiger:
    engineConfig:
      cacheSizeGB: 2
```

TLS CA Zertifikate

Wenn ein CA Zertifikat konfiguriert werden muss.

```
net:  
  tls:  
    CAFile: xxx.pem
```

Dann muss das System Zertifikate entfernt werden.

```
setParameter:  
#  tlsUseSystemCA: true
```

MongoDB Probleme

Platte voll

Lösungsschritte:

- SEAL Software inkl. MongoDB auf allen Rechnern im Cluster runterfahren
- Platte vergrößern lassen
- Zuerst die MongoDB Services wieder hochfahren und warten bis "rs.status()" anzeigt, dass alle Server synchronisiert sind
- Danach erst SEAL Software hochfahren

Eine MongoDB im Cluster synchronisiert nicht mehr

- Synchronisierungsprobleme erkennt man am dauerhaften Status "RECOVERING".
- Nach längerem Ausfall eines MongoDB Service kann es passieren, dass er sich nach dem Hochfahren nicht mehr synchronisieren kann. Hier hilft nur eine vollständige Neusynchronisierung.

Lösungsschritte:

- SEAL Software auf allen Rechnern im Cluster runterfahren
- Betroffenen MongoDB Service herunterfahren
- Alle Dateien im Datenverzeichnis der betroffenen MongoDB (Linux: /opt/seal/data/seal-mongodb, Windows: c:\ProgramData\SEAL Systems\data\seal-mongo) löschen. **Achtung:** nicht das Verzeichnis selbst löschen.
- Betroffene MongoDB wieder starten und so lange warten bis "rs.status()" nicht mehr "RECOVERING" anzeigt. Das kann je nach Datenmenge 10-15 Minuten dauern.
- SEAL Software auf allen Rechnern im Cluster wieder starten

TLS Zertifikat funktioniert nicht

Generell bei Fehlern mit Kundenzertifikaten prüfen:

- Liegen Zertifikat und Key File im PEM Format vor. Andere Formate werden nicht akzeptiert.
- Wurden Zertifikat und Key korrekt in eine gemeinsame Datei kopiert. Die Struktur muss wie folgt aussehen, wobei die Reihenfolge von Zertifikat & Key egal ist:

```
-----BEGIN CERTIFICATE-----  
<Base64 kodiertes Zertifikat>  
-----END CERTIFICATE-----  
-----BEGIN PRIVATE KEY-----  
<Base64 kodierter Key>  
-----END PRIVATE KEY-----
```

- Gibt es ein CA Zertifikat und wurde es korrekt konfiguriert, siehe [TLS CA Zertifikate](#)
- Logmeldungen prüfen. Die Fehlermeldungen im [MongoDB Logfile](#) sind in der Regel aussagekräftig und weisen sofort in die richtige Richtung.

Beispiele für Fehlermeldungen:

"The provided SSL certificate is expired or not yet valid.": Das Zertifikat ist nicht (mehr) gültig, es muss ein neues Zertifikat generiert werden.

"The use of TLS without specifying a chain of trust is no longer supported ...": es fehlt ein CA Zertifikat, siehe [TLS CA Zertifikate](#).

"The use of both a CA File and the System Certificate store is not supported.": "CAFile" und "tlsUseSystemCA" sind vorhanden, siehe [TLS CA Zertifikate](#).

"Can not set up PEM key file.": die Datei auf die "certificateKeyFile" verweist ist nicht korrekt. Hier gib tes mehrere Möglichkeiten. Entweder die Datei enthält nur das Zertifikat oder nur den Key, oder Zertifikat/Key ist generell kaputt.

ReplicaSet nicht initialisiert

Wenn ein "rs.status()" die Meldung "MongoServerError: no replset config has been received" liefert, wurde "rs.initiate()" vergessen.

MongoDB nützliche Kommandos

MongoDB Shell Aufruf

Windows:

```
& "C:\Program Files\mongosh\mongosh.exe" --tls --tlsAllowInvalidCertificates
```

Linux:

```
mongosh --tls --tlsAllowInvalidCertificates
```

Batch Kommandos generell

```
--eval "<Kommando>"
```

ReplicaSet prüfen

```
rs.status()
```

Vollständige Zeile für Windows:

```
& "C:\Program Files\mongosh\mongosh.exe" --tls --tlsAllowInvalidCertificates  
--eval "rs.status()"
```

Vollständige Zeile für Linux:

```
mongosh --tls --tlsAllowInvalidCertificates --eval "rs.status()"
```

Interaktive Kommandos

show dbs zeigt alle Datenbanken an. Beipielausgabe:

admin	128.00 KiB
config	364.00 KiB
local	234.54 MiB
spooler-actions	8.00 KiB
spooler-configs	108.00 KiB
spooler-jobs	30.33 MiB
spooler-locks	108.00 KiB
spooler-notifications	116.00 KiB
spooler-preprocess	108.00 KiB
spooler-printers	484.00 KiB
spooler-remote-sites	72.00 KiB
spooler-ui	96.00 KiB

use <db name> wechselt in die angegebene Datenbank . Beispiel:

```
use spooler-jobs
```

db.dropDatabase() löscht die mit **use** aktuell ausgewählte Datenbank. **Nur im Notfall benutzen!**

NATS

NATS Config

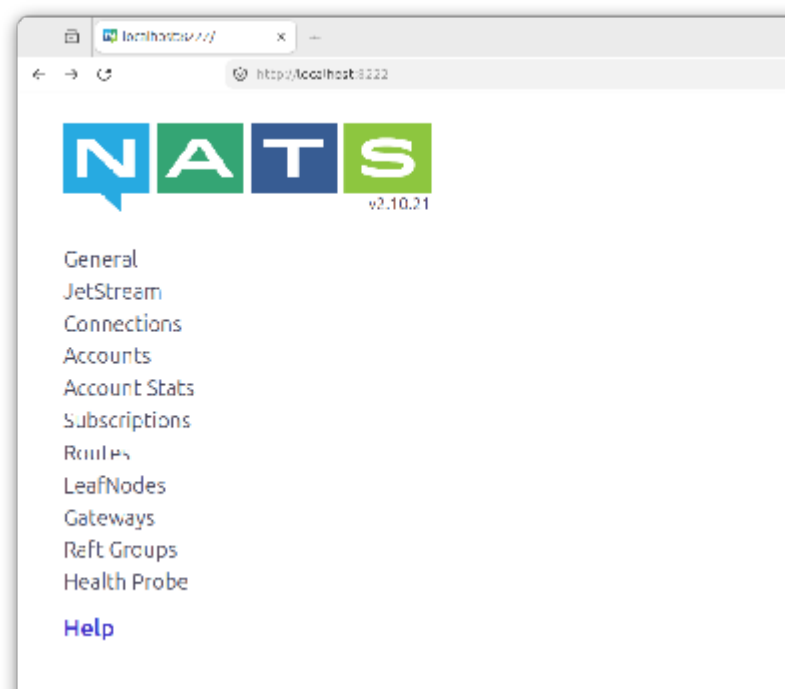
Monitoring einschalten

Zum Prüfen des NATS Status kann der Monitoring Port in der "nats.conf" eingeschaltet werden. Die notwendige Zeile ist bereits als Kommentar enthalten.

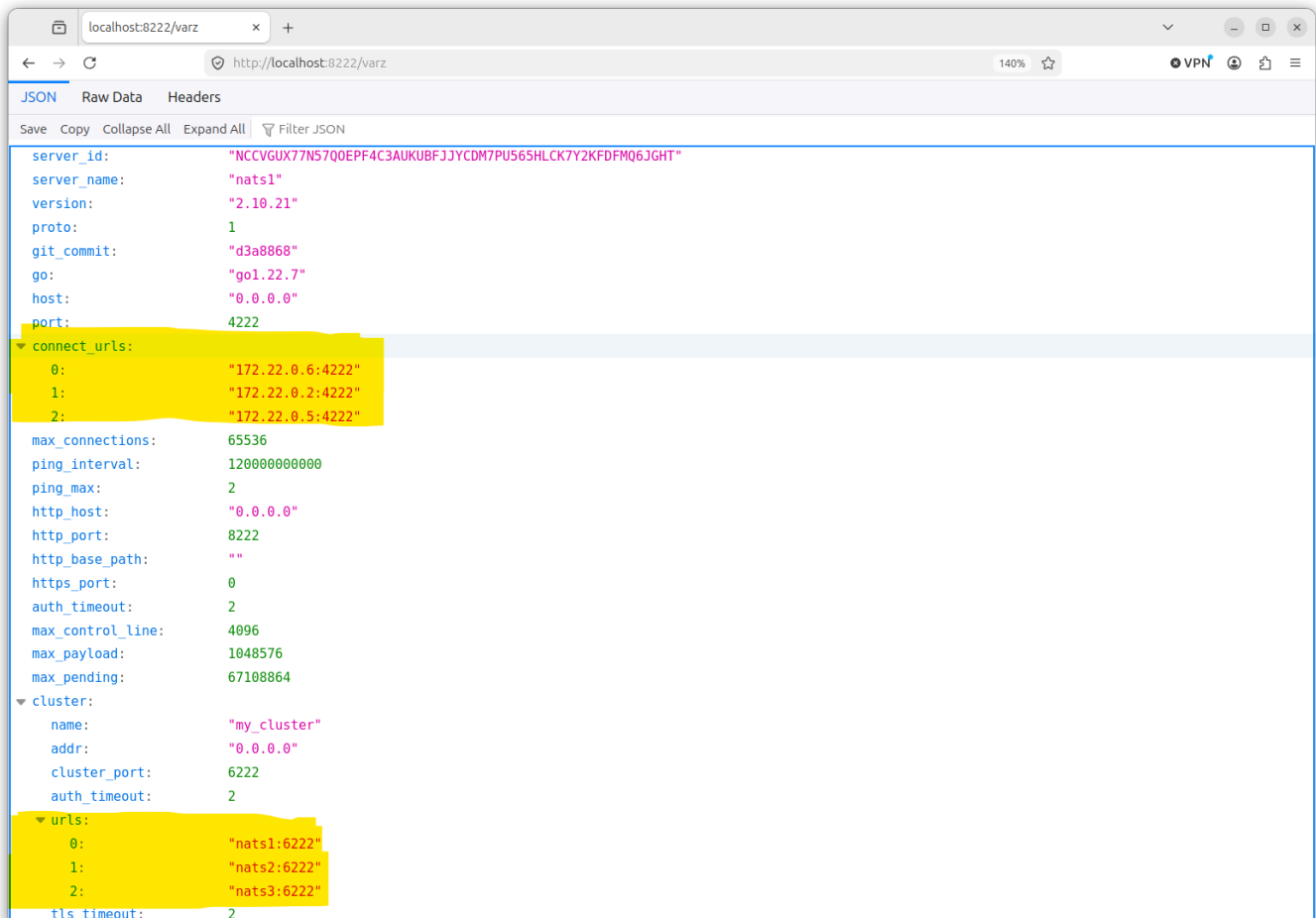
```
# HTTP monitoring port  
monitor_port: 8222
```

Nach NATS Neustart kann im Browser der Status angezeigt werden. Falls auf dem Server kein Browser vorliegt und von außen darauf zugegriffen werden muss, nicht vergessen den Port in der Firewall frei zu schalten, falls noch nicht erledigt.

Die Startseite des Monitoring sieht so aus:



Cluster Verbindungen unter dem Punkt "General":



Debug und Trace Meldungen einschalten

In der Regel nicht benötigt, Fehler stehen auch im normalen Logfile.

Trace nur im äußersten Notfall, die Logdateien wachsen damit sehr schnell auf eine enorme Größe an.

```
debug: true
trace: true
```

NATS Probleme

Keine bekannten Probleme!

Aber: Es gab im seal-co-notifier ein Problem, dort wurde auf den NATS KV Store nicht korrekt zugegriffen. Ist mit seal-co-notifier ab Version 5.1.2 und PLOSSYS Output Engine 7.4.0 behoben.

NATS nützliche Kommandos

Verbindung zum Service prüfen

```
nats server check connection -s nats://<hostname>:4222
```